



Universidad
de Jaén

Departamento de Informática

Prácticas de Estructuras de Datos

Grado en Ingeniería en Informática

Curso 2026/2027

Práctica 1. Implementación de vector dinámico mediante plantillas y operadores en C++

Sesiones de prácticas: 2

Objetivos

- Implementar la clase `VDinamico<T>` utilizando **patrones de clase y excepciones**.
- Programa de prueba para comprobar su correcto funcionamiento.

Descripción de la EEDD

Implementar la clase `VDinamico<T>` para que tenga toda la funcionalidad del vector dinámico descrita en la Lección 4, utilizando patrones de clase (Lección 3) y excepciones. Los métodos a implementar serán los siguientes:

- Constructor por defecto `VDinamico<T>`, iniciando el tamaño físico a 1 y el lógico a 0.
- Constructor dando un tamaño lógico inicial, `VDinamico<T>(unsigned int tamlog, T &dato)`, que rellena el vector con `tamlog` elementos de tipo `T`. El tamaño físico será siempre la potencia de 2 igual o inmediatamente superior a `tamlog` (tamaño lógico).
- Constructor copia `VDinamico<T>(const VDinamico<T>& origen)`.
- Constructor de copia parcial `VDinamico<T>(const VDinamico<T>& origen, unsigned int posicionInicial, unsigned int numElementos)`. Que crea un vector de tamaño lógico `numElementos` a partir de `posicionInicial` del vector `origen`. El tamaño físico también debe ser una potencia de 2.
- Operador de asignación (`=`).
- Operador `[]` para acceder a un dato para lectura/escritura.
- Insertar un dato en una posición: `void insertar(const T& dato, unsigned int pos = UINT_MAX)`. Si no se indica la posición (valor `UINT_MAX`) entonces la inserción se realiza al final del vector. Esta operación siempre incrementa el tamaño lógico en 1.
- Eliminar un dato de una posición intermedia en $O(n)$: `T borrar(unsigned int pos = UINT_MAX)`. Si no se indica la posición (valor `UINT_MAX`) entonces se elimina el último dato del vector.
- Ordenar el vector de menor a mayor. Se puede utilizar la función `sort` de `<algorithm>`: **`void ordenar()`**. Para ello hay que dotar a la clase `T` del **`operator<`** y el **`operator==`** (en este caso la clase `Especie` debe tener el operador implementado).

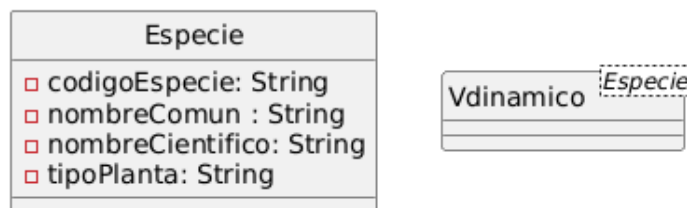
- Búsqueda binaria de un elemento. Dado un dato de tipo T se localiza con la búsqueda dicotómica asumiendo que los datos del vector están ordenados. Se devuelve la posición donde se localiza el dato. `int busquedaDicotomica(const T& dato).`
- `unsigned int tamlog()` para obtener el tamaño (lógico) del vector.
- El destructor correspondiente.

Programa de prueba: Creación de un vector dinámico de especies de árboles

En este curso diseñaremos e implementaremos un sistema de gestión para la masa arbórea y las zonas verdes de la ciudad. El objetivo: digitalizar el inventario de parques y especies para agilizar el control de mantenimiento.

En esta primera práctica utilizaremos la clase Especie representando cada una de las Especies a las que puede pertenecer un determinado árbol según la Unión Internacional para la Conservación de la Naturaleza (IUCN). Estos datos se encuentran disponibles en el fichero **arbolado-especies.csv** adjunto al enunciado (una línea por especie). Estos datos se almacenarán en un **vector dinámico**, concretamente, en el contenedor `VDinamico<Especie>`. Los valores de los atributos aparecen en el fichero en el mismo orden que en el UML, y corresponden con los cuatro atributos de la clase. El código de lectura de fichero se proporciona junto al enunciado de la práctica, encapsulado en la clase `LectorCSV`. Especie se ha implementado como `struct`, con el objetivo de que el alumno/a **modifique el código base e implemente la clase Especie**.

La clase Especie contendrá exclusivamente los **constructores**, **operadores** y **getters/setters** necesarios para la práctica. El **criterio de ordenación** de Especie será **por codigoEspecie**.

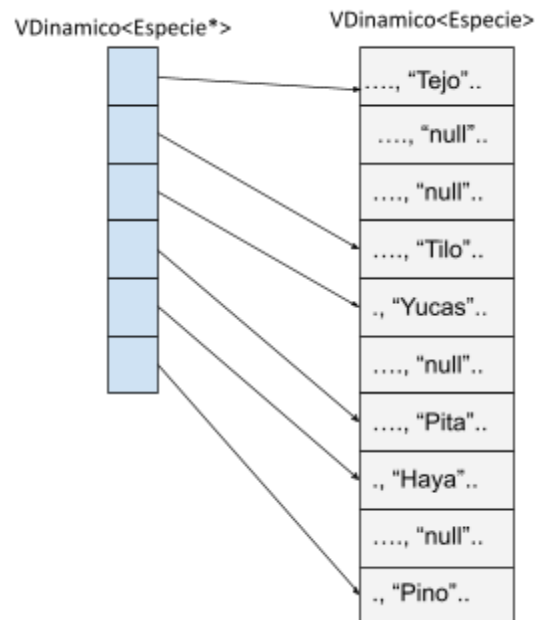


La prueba implementada en la función `main ()` consistirá en:

- Implementar el vector dinámico `VDinamico<T>` y la clase Especie atendiendo a las especificaciones anteriores de las Lecciones 3 y 4 del temario.
- Instanciar el vector con todos los objetos de tipo Especie contenidos en el fichero adjunto: **arbolado_especies.csv**. Mostrar los identificadores de los primeros 50 elementos del vector.
- Ordenar el vector de menor a mayor (por el `codigoEspecie`) y mostrar los datos de las primeras 50 especies.
- Una vez ordenado el vector, buscar en $O(\log n)$ las especies con los códigos: CTA, DMD, HCN, NDOF y JAX, mostrando su posición en el vector, teniendo en cuenta que pueden no existir.
- El fichero adjunto tiene muchos datos nulos en “Nombre común”. Crear un nuevo vector dinámico de punteros a objetos de tipo `Especie` que apunten al vector original pero solo si tiene un

valor no nulo en el nombre común como se observa en la figura (null significa cadena vacía en la figura). Mostrar en pantalla el tamaño de dicho vector y los 50 primeros datos.

```
VDinamico<Especie*> getEspNComun(VDinamico<Especie> &vEspecies);
```



- Aquellos que hagáis las prácticas **por parejas** debéis además:
 - o Ordenar el vector por el string “codigoEspecie” directamente en el código de prueba con el método burbuja (no incluir esta operación en `VDinamico<T>`). Mostrar los últimos 50 elementos en orden inverso.
 - o Una vez ordenados, averiguar cuántas especies contienen una palabra especificada por parámetro **en la primera palabra de su nombre científico**. Por ejemplo, especificando ‘Jasminum’, el número de especies que contienen esta palabra en su nombre científico es 6 (Jasminum nudiflorum, Jasminum fruticans...). Implementar una función que devuelva un vector de punteros hacia las especies que cumplan la condición. El resultado de la prueba debe ser un valor numérico que aparecerá por pantalla.

Estilo y requerimientos del código:

1. El código debe ser capaz de ejecutarse con la función `main()` que se adjunta.
2. El código debe ser claro, tener un estilo definido y estar perfectamente indentado, para ello se pueden seguir algunos de los estilos preestablecidos para el lenguaje C++ (<http://geosoft.no/development/cppstyle.html>).
3. Deben comprobarse todas los posibles errores y situaciones de riesgo que puedan ocurrir (desbordamientos de memoria, parámetros con valores no válidos, etc.) y lanzar las excepciones correspondientes, siempre que tenga sentido. Leer el tutorial de excepciones disponible en el repositorio de la asignatura en docencia virtual.

4. Se valorará positivamente la calidad general del código: claridad, estilo, ausencia de redundancias, etc.